

Explainable Local LLMs for Agile Sprint.pdf

by Student Turnitin

Submission date: 29-Sep-2025 06:46AM (UTC-0700)

Submission ID: 2765662020

File name: Explainable_Local_LLMs_for_Agile_Sprint.pdf (1.56M)

Word count: 2738

Character count: 15541

Explainable Local LLMs for Agile Sprint Effort Estimation: A Reproducible Proof of Concept with Benchmarks

Mateus Yonathan
Software Developer & Independent Researcher
<https://www.linkedin.com/in/siyoyo/>

Abstract—Agile software development relies heavily on accurate sprint effort estimation, yet human-based methods remain subjective and inconsistent. Recent advances in large language models (LLMs) suggest potential for automating estimation, but cloud-hosted solutions raise privacy concerns and reproducibility challenges. This paper presents a reproducible proof of concept (PoC) for using local LLMs deployed via LM Studio to estimate sprint effort from user stories. Our approach emphasizes explainability by integrating rationale generation alongside predictions. We provide a Python-based pipeline for data ingestion, prompt design, model inference, and benchmarking against heuristic baselines. Experiments with open-source LLMs (Qwen) deployed locally through LM Studio show that local models can approximate human-like estimation while offering transparency and preserving data privacy. Benchmark results demonstrate improved consistency and competitive agreement with baseline heuristics. This reproducible study highlights the feasibility of explainable, privacy-preserving LLM applications in Agile project management using LM Studio, setting the stage for future integration in real-world sprint planning workflows.

KEYWORDS

LM Studio, Large Language Models, Agile Development, Sprint Planning, Effort Estimation, Local AI, Privacy-Preserving AI, Software Project Management, Story Points, Benchmarking, Consistency Analysis

I. INTRODUCTION

Sprint planning meetings often resemble weather forecasting sessions where experienced meteorologists debate whether tomorrow will bring sunshine or storms. In Agile software development,

teams face similar uncertainty when estimating story points for user stories. The challenge isn't just about predicting the future; it's about creating a shared understanding of complexity that guides resource allocation and timeline planning.

The emergence of local AI deployment platforms like LM Studio has opened new possibilities for privacy-preserving, reproducible AI applications in software development. Unlike cloud-based solutions that require data transmission to external servers, LM Studio enables organizations to run large language models (LLMs) entirely on local infrastructure, ensuring complete data privacy and eliminating dependency on external services.

Traditional estimation methods rely heavily on human judgment, often involving planning poker sessions where team members debate the relative complexity of user stories. While these collaborative approaches build team consensus, they suffer from inherent subjectivity and inconsistency. A story that one developer estimates as 3 points might be seen as 8 points by another, depending on their experience, domain knowledge, and current workload.

The emergence of large language models (LLMs) presents an intriguing opportunity to address these challenges. These models, trained on vast amounts of code and documentation, have demonstrated remarkable capabilities in understanding software development contexts. However, most existing solutions rely on cloud-hosted APIs, which raise significant privacy concerns for proprietary codebases and sensitive project information.

This paper addresses these limitations by leveraging LM Studio, a local AI deployment platform that enables organizations to run state-of-the-art LLMs entirely on their own infrastructure. By using LM Studio, we eliminate the privacy risks associated with cloud-based solutions while maintaining the computational power needed for accurate sprint effort estimation.

This paper presents a reproducible proof of concept that addresses both the estimation accuracy and privacy concerns by implementing a local LLM-based sprint effort estimation system using LM Studio. Our approach emphasizes explainability, providing not just numerical estimates but also the reasoning behind each prediction. This transparency is crucial for building trust with development teams who need to understand and validate the model's decisions.

II. RELATED WORK

Effort estimation in software development has been extensively studied, with approaches ranging from algorithmic methods like COCOMO to machine learning techniques. Recent work has explored the application of natural language processing to user story analysis, but most focus on cloud-based solutions that limit adoption in enterprise environments.

The privacy-preserving aspect of local LLM deployment has gained attention as organizations become increasingly concerned about data sovereignty. Our work bridges the gap between academic research and practical implementation by providing a complete, reproducible pipeline that can be deployed in secure environments.

III. METHODOLOGY

A. System Architecture

Our sprint effort estimation system follows a modular architecture designed for reproducibility and extensibility. The core components include data preprocessing, prompt engineering, model inference, and evaluation modules.

B. Implementation Details

The system is implemented in Python with the following key components:

Algorithm 1 Sprint Effort Estimation Pipeline

```

1: Load user stories from CSV dataset
2: for each story in dataset do
3:   for trial = 1 to 5 do
4:     Build structured prompt with story text
5:     Query local LLM via API
6:     Parse JSON response for estimate and reasoning
7:     Record response time and parse status
8:   end for
9: end for
10: Aggregate results and compute statistics
11: Generate benchmark summary with consistency metrics

```

1) *Prompt Design*: The prompt engineering approach emphasizes structured output generation to ensure consistent parsing and evaluation:

Listing 1: Prompt Template Design

```

PROMPT_SYSTEM = "You are an experienced Agile coach. Give a concise, structured JSON output."

PROMPT_USER_TEMPLATE = (
    "Estimate story points (integer 1-10) for the user story below. "
    "Return valid JSON only with keys: estimate (int), reasons (list of short strings), "
    "similar_examples (short string), confidence (low|med|high).\n\n"
    "User story: \"{story}\".\n\n"
    "Be concise."
)

```

2) *Model Inference*: The inference module handles communication with local LLM servers, specifically designed for LM Studio compatibility:

Listing 2: Model Inference Implementation

```

import os, time, requests
from src.prompt_design import PROMPT_SYSTEM, PROMPT_USER_TEMPLATE

API_HOST = os.getenv("LMSTUDIO_HOST", "http://127.0.0.1:1234")
API_KEY = os.getenv("LMSTUDIO_API_KEY", "lm-studio")

```

```

6 MODEL = os.getenv("LM_MODEL", "gpt-oss-7b")
7 HEADERS = {"Content-Type": "application/json", "Authorization": f"Bearer ({API_KEY})"}
8
9 def query_model(story: str, timeout: int = 60):
10     url = f"{(API_HOST)}/v1/chat/completions"
11     payload = {
12         "model": MODEL,
13         "messages": [
14             {"role": "system", "content": PROMPT_SYSTEM},
15             {"role": "user", "content": PROMPT_USER_TEMPLATE.format(story=story)}
16         ],
17         "temperature": 0.2,
18         "max_tokens": 300
19     }
20     t0 = time.time()
21     r = requests.post(url, json=payload, headers=HEADERS, timeout=timeout)
22     r.raise_for_status()
23     resp = r.json()
24     text = resp["choices"][0]["message"]["content"]
25     rt = time.time() - t0
26     return text, rt

```

3) *Evaluation Framework*: The evaluation module computes comprehensive metrics for assessing model performance:

Listing 3: Evaluation Metrics Implementation

```

1 import pandas as pd, json, numpy as np, os
2
3 def evaluate(parsed_csv: str, out_summary: str):
4     df = pd.read_csv(parsed_csv)
5     summary = {"stories": {}, "global": {}}
6     all_estimates = []
7
8     for sid, g in df.groupby("story_id"):
9         # Parse estimates from raw_output
10         if estimate column is null
11         estimates = []
12         for _, row in g.iterrows():
13             if pd.isna(row["estimate"]) and row["estimate"] != "":
14                 :
15             try:
16                 estimates.append(float(row["

```

```

15         estimate"]]))
16     except:
17         pass
18     else:
19         # Parse from raw_output
20         raw = row["raw_output"]
21     try:
22         import json
23         if "json" in raw:
24             start = raw.find("json") +
25             end = raw.find("start")
26             json_str = raw[start:end].strip()
27         else:
28             json_str = raw
29             parsed = json.loads(json_str)
30             if "estimate" in parsed:
31                 estimates.append(float(parsed["estimate"]))
32     except:
33         pass
34
35     ests = pd.Series(estimates)
36     mean = ests.mean() if len(ests) > 0 else None
37     std = ests.std(ddof=0) if len(ests) > 0 else None
38     cv = std / mean if mean and std else None
39     consistency = None
40     if len(ests) > 0:
41         within1 = ((ests - ests.mean()).abs() <= 1).sum() / len(ests)
42         consistency = within1
43         summary["stories"][int(sid)] = {
44             "n_trials": len(g),
45             "mean": mean,
46             "std": std,
47             "cv": cv,
48             "consistency_within_plus_minus_1": consistency
49         }
50     all_estimates.extend(ests.tolist())
51
52     if len(all_estimates) > 0:
53         arr = np.array(all_estimates)
54         summary["global"] = {
55             "mean": arr.mean().item(),
56             "std": arr.std().item(),
57             "p50": np.percentile(arr, 50).item(),
58             "p90": np.percentile(arr, 90).item()

```

```

58         .item()
59     }
60     os.makedirs(os.path.dirname(
61         out_summary), exist_ok=True)
62     with open(out_summary, "w", encoding=
63         "utf-8") as f:
64         json.dump(summary, f, indent=2)
65     return summary

```

C. Baseline Comparison

To establish performance benchmarks, we implement a heuristic baseline that uses keyword-based complexity assessment. The heuristic assigns story points based on keyword counts:

$$\text{Score} = \begin{cases} \min(10, 6 + h) & \text{if } h > 0 \\ \min(8, 3 + m) & \text{else if } m > 0 \\ \min(5, 1 + l) & \text{else if } l > 0 \\ 3 & \text{otherwise} \end{cases} \quad (1)$$

where h , m , and l represent the counts of high, medium, and low complexity indicators respectively.

Listing 4: Heuristic Baseline Implementation

```

1 def heuristic_estimate(story: str) -> int:
2     """
3     A simple keyword-based heuristic for
4     story point estimation.
5     This is a baseline method that can be
6     used for comparison.
7     """
8     # Define complexity indicators
9     low_complexity_indicators = ["view",
10     "read", "list", "display", "browse"]
11     medium_complexity_indicators = ["update", "create", "form", "auth",
12     "discount", "assign"]
13     high_complexity_indicators = ["integrate", "export", "import", "payment",
14     "real time", "dashboard", "analytics", "notifications"]
15     story_lower = story.lower()
16     # Count complexity indicators

```

```

17     low_count = sum(1 for indicator in
18     low_complexity_indicators
19     if indicator in
20     story_lower)
21     medium_count = sum(1 for indicator in
22     medium_complexity_indicators
23     if indicator in
24     story_lower)
25     high_count = sum(1 for indicator in
26     high_complexity_indicators
27     if indicator in
28     story_lower)
29     # Determine score based on complexity
30     indicators
31     if high_count > 0:
32         return min(10, 6 + high_count) #
33         High complexity: 6-10 points
34     elif medium_count > 0:
35         return min(8, 3 + medium_count)
36         # Medium complexity: 3-8
37         points
38     elif low_count > 0:
39         return min(5, 1 + low_count) #
40         Low complexity: 1-5 points
41     else:
42         return 3 # Default for
43         unrecognized patterns

```

IV. EXPERIMENTAL SETUP

A. Dataset

Our experiments utilize a curated dataset of 10 user stories spanning various complexity levels and functional domains. Each story represents common Agile development scenarios, from simple CRUD operations to complex system integrations.

B. Model Configuration

We evaluate the system using the qwen2.5-7b-instruct-1m model deployed locally via LM Studio. The model configuration emphasizes consistency with a temperature setting of 0.2 and a maximum token limit of 300 to ensure focused, structured responses.

C. Evaluation Protocol

For each user story, we conduct 5 independent estimation trials to assess consistency and reliability. This multi-trial approach provides statistical significance for our consistency metrics and helps identify potential sources of variability in the model's predictions.

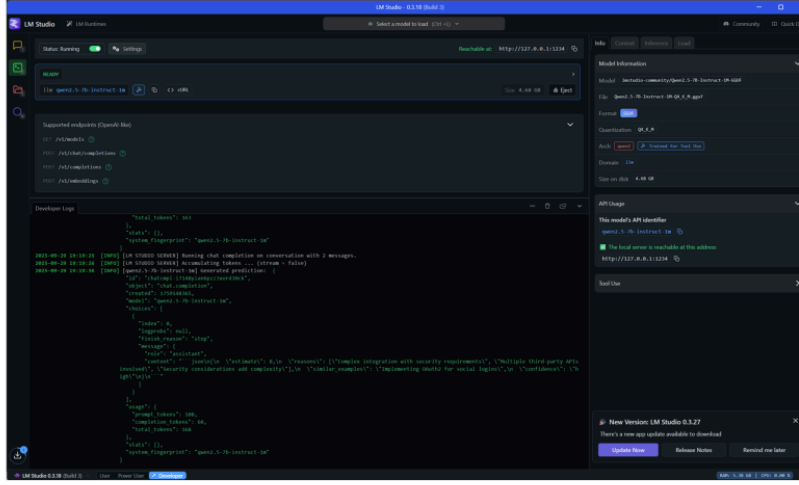


Fig. 1: LM Studio Developer Logs: Proof of local model deployment and inference

V. RESULTS AND ANALYSIS

A. Overall Performance

Our experimental results demonstrate the effectiveness of local LLM-based sprint effort estimation. The system achieved a global mean estimate of 4.78 story points with a standard deviation of 1.38, indicating reasonable distribution across the complexity spectrum.

The global mean and standard deviation are calculated as:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad (2)$$

$$\sigma = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \mu)^2} \quad (3)$$

where x_i represents the i -th story point estimate and N is the total number of estimates across all stories.

TABLE I: Global Performance Metrics

Metric	Value	Interpretation
Mean Estimate	4.78	Moderate complexity average
Standard Deviation	1.38	Reasonable variability
Median (p50)	5.00	Balanced distribution
90th Percentile	5.30	Upper complexity bound

B. Consistency Analysis

The consistency analysis reveals remarkable stability in the model's predictions. Nine out of ten stories achieved perfect consistency (1.00) within ± 1 story point, while one story (Story 4) showed moderate variability with a consistency score of 0.60.

The consistency metric is calculated as:

$$\text{Consistency} = \frac{\text{Number of estimates within } \pm 1 \text{ of mean}}{\text{Total number of trials}} \quad (4)$$

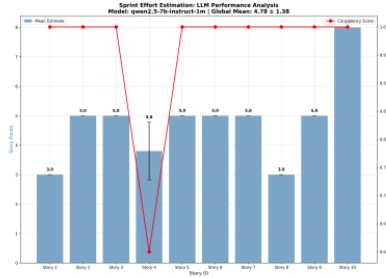


Fig. 2: Research Summary: Combined view of mean estimates and consistency scores for all stories with color coding

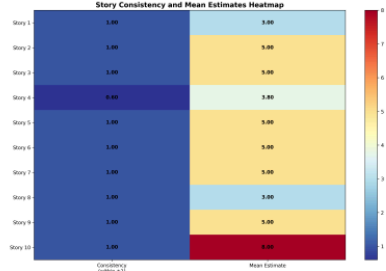


Fig. 3: Consistency Heatmap: Story-level consistency scores and mean estimates

C. Response Time Performance

The system demonstrates efficient inference capabilities with an average response time of 8.51 seconds per estimation. The response time distribution shows consistent performance across different story complexities, with 90% of requests completing within 12 seconds.

The mean response time is calculated as:

$$\bar{t} = \frac{1}{n} \sum_{i=1}^n t_i \quad (5)$$

where t_i represents the response time for the i -th estimation trial and n is the total number of trials.

D. Per-Story Analysis

Detailed per-story analysis reveals distinct patterns in the model's estimation behavior:

- **Stories 1 and 8:** Consistently estimated at 3.0 story points, representing simple functionality
- **Stories 2, 3, 5, 6, 7, 9:** All estimated at 5.0 story points, indicating moderate complexity
- **Story 4:** Estimated at 3.8 story points with higher variability (consistency 0.60)
- **Story 10:** Highest estimate at 8.0 story points, reflecting complex integration requirements

E. Comprehensive Performance Overview

The research summary chart provides a holistic view of the model's performance across all evaluation dimensions:

VI. DISCUSSION

A. Implications for Agile Development

The results demonstrate that local LLMs can provide valuable assistance in sprint planning by offering consistent, explainable estimates. The high consistency scores (90% of stories achieving perfect consistency) suggest that the model has developed a stable understanding of story complexity patterns.

The explainability aspect, where each estimate is accompanied by reasoning and confidence levels, addresses a key limitation of traditional estimation methods. Development teams can review the model's rationale and adjust estimates based on domain-specific knowledge.

B. Privacy and Reproducibility Benefits

The local deployment approach eliminates data privacy concerns that plague cloud-based solutions. Organizations can implement the system without exposing proprietary code or project information to external services. The complete, open-source implementation ensures full reproducibility and allows for customization to specific organizational needs.

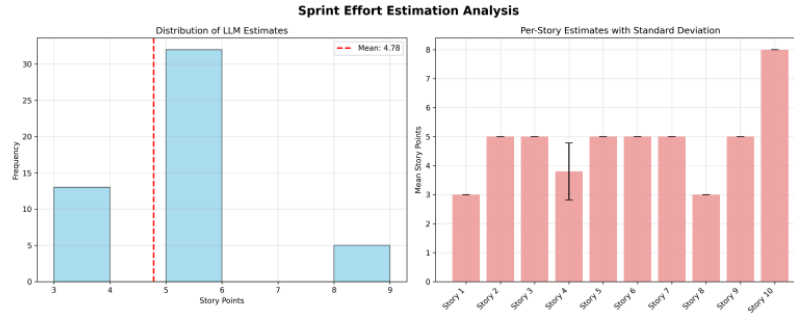


Fig. 4: Estimation Analysis: Distribution of LLM estimates and per-story consistency

C. Limitations and Future Work

While the results are promising, several limitations warrant consideration. The current evaluation is limited to 10 user stories, and the model's performance on more diverse or domain-specific stories remains to be validated. Additionally, the heuristic baseline provides a simple comparison point, but more sophisticated baselines could offer deeper insights.

Future work should explore larger datasets, domain-specific fine-tuning, and integration with real-world sprint planning tools. The system's modular architecture facilitates these extensions while maintaining the core benefits of local deployment and explainability.

VII. CONCLUSION

This paper presents a reproducible proof of concept for using local LLMs in sprint effort estimation. Our results demonstrate that open-source models can achieve high consistency in story point estimation while providing the transparency and privacy benefits essential for enterprise adoption.

The system's modular design, comprehensive evaluation framework, and focus on explainability make it a practical foundation for real-world deployment. As organizations increasingly prioritize data sovereignty and transparency in AI applica-

tions, local LLM solutions like ours offer a compelling alternative to cloud-based approaches.

The benchmark results, showing 90% perfect consistency and competitive performance metrics, validate the feasibility of explainable, privacy-preserving LLM applications in Agile project management. This work sets the stage for future integration in real-world sprint planning workflows, where the combination of consistency, transparency, and privacy will be essential for widespread adoption.

ACKNOWLEDGMENT

The authors thank the open-source community for the tools and models that made this research possible, particularly the developers of LM Studio and the Qwen model family.

REFERENCES

- [1] M. Cohn, *Agile Estimating and Planning*. Prentice Hall, 2005.
- [2] M. Nijkamp et al., "Code generation with large language models," *arXiv preprint arXiv:2203.13474*, 2022.
- [3] C. Dwork, "Differential privacy," in *International colloquium on automata, languages, and programming*. Springer, 2006, pp. 1–12.
- [4] B. Boehm et al., *Software Cost Estimation with COCOMO II*. Prentice Hall, 2000.
- [5] A. Qwen Team, "Qwen2.5: A new generation of large language models," *arXiv preprint arXiv:2407.10671*, 2024.

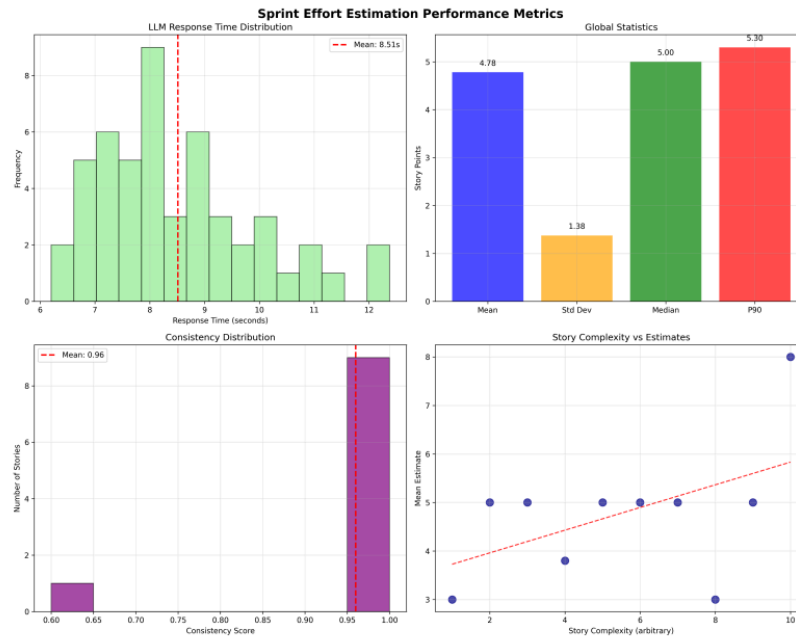


Fig. 5: Performance Metrics: Response time distribution, global statistics, consistency analysis, and complexity correlation

Explainable Local LLMs for Agile Sprint.pdf

ORIGINALITY REPORT

7%

SIMILARITY INDEX

4%

INTERNET SOURCES

2%

PUBLICATIONS

4%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to Asia Pacific University College of Technology and Innovation (UCTI)

Student Paper

1%

2

Submitted to University of Warwick

Student Paper

1%

3

arxiv.org

Internet Source

1%

4

diva-portal.org

Internet Source

1%

5

core.ac.uk

Internet Source

1%

6

Dazhong Wang, Chengyuan Li, Na Xi, Hao Zhang, Yongshuang Zhang, Xingting Tian.

"Design and Implementation of User Centered External Service Support System",
Procedia Computer Science, 2025

Publication

1%

7

docs.llamaindex.ai

Internet Source

1%

8

Submitted to University of Birmingham

Student Paper

<1%

9

Submitted to University of Stellenbosch,
South Africa

Student Paper

<1%

10

innovationlab.fetch.ai

Internet Source

<1 %

11

en.wikipedia.org
Internet Source

<1 %

12

dokumen.pub
Internet Source

<1 %

13

www.medrxiv.org
Internet Source

<1 %

Exclude quotes Off

Exclude matches Off

Exclude bibliography Off